

Deepseek技术浅析（二）：大语言模型

原创

爱研究的小牛

于 2025-01-29 20:57:13 发布

阅读量3.1k

收藏 35

点赞数 20

分类专栏：

大模型

文章标签：

语言模型

人工智能

自然语言处理

AIGC

机器学习

CC 4.0 BY-SA版权

DeepSeek 作为一家致力于人工智能技术研发的公司，其大语言模型（LLM）在架构创新、参数规模扩展以及训练方法优化等方面都达到了行业领先水平。

一、基于 Transformer 架构的创新

1.1 基础架构：Transformer 的回顾

Transformer 架构是当前最先进的大语言模型架构，其核心机制是**自注意力机制（Self-Attention Mechanism）**，能够有效捕捉序列中长距离的依赖关系。以下是 Transformer 的主要组件及其工作原理：

(1) 编码器 (Encoder)

• 多头自注意力机制 (Multi-Head Self-Attention)：

- **目的:** 捕捉输入序列中不同位置之间的依赖关系。
- **工作原理:**

- 将输入向量 x 分别线性变换为查询 (Query, Q)、键 (Key, K) 和值 (Value, V) 矩阵：

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

其中, W_Q, W_K, W_V 是可训练的权重矩阵。

- 计算注意力权重：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中, d_k 是键的维度，用于缩放注意力权重，防止梯度消失。

- 多头机制: 将输入向量分割成多个头 (head)，每个头独立计算注意力，然后将多个头的输出拼接起来，并通过线性变换得到最终输出。

- **前馈神经网络 (Feed-Forward Neural Network, FFN) :**
 - 对每个位置的表示进行非线性变换。
 - 通常由两个线性变换和一个激活函数（如 ReLU）组成：

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

其中, W_1, W_2, b_1, b_2 是可训练的参数。

(2) 解码器 (Decoder)

- 除了多头自注意力机制和前馈神经网络外，解码器还包含一个**编码器-解码器注意力机制 (Encoder-Decoder Attention)**，用于捕捉编码器输出与解码器输入之间的关系。
- 在训练过程中，使用**掩码 (Masking)** 机制，防止模型在预测当前词时看到未来的词。

1.2 DeepSeek 的创新与改进

DeepSeek 在 Transformer 基础上进行了多项创新和改进，以提升模型性能、效率和可扩展性。

(1) 稀疏注意力机制 (Sparse Attention Mechanisms)

- **问题:** 传统的自注意力机制计算复杂度为 $O(n^2)$ ，其中 n 是序列长度。当处理长序列时，计算量和内存需求会急剧增加。
- **解决方案:**
 - **局部注意力 (Local Attention) :**

- **原理:** 只计算相邻位置的注意力权重，例如只计算每个位置与前后 k 个位置的注意力。
 - **公式表示:**

$$LocalAttention(Q, K, V) = \text{softmax} \left(\frac{QK_{local}^T}{\sqrt{d_k}} \right) V_{local}$$

其中, K_{local}, V_{local} 是局部键和值矩阵。

- **优点:** 降低计算复杂度，适用于局部依赖性强的任务。

- **缺点:** 无法捕捉长距离依赖关系。
- **块状注意力 (Blockwise Attention) :**
 - **原理:** 将序列划分为多个块，只计算块内和块间部分位置的注意力权重。
 - **优点:** 在一定程度上保留长距离依赖关系，同时降低计算复杂度。
 - **缺点:** 块间信息传递效率较低。
- **随机注意力 (Random Attention) :**
 - **原理:** 随机选择部分位置计算注意力权重。
 - **优点:** 增加模型的多样性，可能捕捉到一些意想不到的依赖关系。
 - **缺点:** 随机性可能导致模型不稳定。
- **组合策略:**
 - DeepSeek 采用多种稀疏注意力机制的组合，例如：
 - **局部 + 块状:** 先使用局部注意力捕捉局部依赖关系，再使用块状注意力捕捉长距离依赖关系。
 - **局部 + 随机:** 在局部注意力的基础上，添加随机注意力，增加模型的多样性。
- **优势:**
 - 显著降低计算复杂度和内存需求。
 - 在保持模型性能的同时，提高训练和推理效率。

(2) 混合专家模型 (Mixture of Experts, MoE)

- **问题:** 大规模模型参数数量庞大，但并非所有参数在所有任务或输入上都发挥作用，导致计算资源浪费。
- **解决方案:**
 - **专家网络 (Expert Networks) :**
 - DeepSeek 将模型划分为多个专家子模型，每个专家网络是一个独立的神经网络，可以是前馈神经网络、注意力机制等。

- 每个专家网络可以专注于不同的任务或领域，例如：
 - **语言专家**: 处理语言相关的任务。
 - **知识专家**: 存储和利用知识库信息。
 - **推理专家**: 执行推理任务。

• **门控网络（Gating Network）：**

- 根据输入内容，动态选择激活哪些专家网络。
- **公式表示：**

$$\text{Gate}(x) = \text{softmax}(xW_g + b_g)$$

其中, W_g, b_g 是门控网络的参数。

- **作用**: 根据输入的不同，分配不同的计算资源给不同的专家网络。

• **输出组合：**

- 将激活的专家网络的输出进行加权组合，得到最终输出。
- **公式表示：**

$$y = \sum_{i=1}^N \text{Gate}_i(x) \cdot \text{Expert}_i(x)$$

其中, N 是专家网络的数量, $\text{Expert}_i(x)$ 是第 i 个专家网络的输出。

• **优势：**

- 提高模型的参数利用率和计算效率。
- 允许模型根据不同的输入动态调整其行为，提高模型的灵活性和适应性。

(3) 多头潜在注意力（Multi-Head Latent Attention, MLA）

- **问题**: 在推理过程中，注意力机制的计算量仍然较大，尤其是当模型规模较大时。

• 解决方案:

• 低秩分解:

- 将键和值矩阵分解为低秩矩阵，从而减少存储和计算需求。

• 公式表示:

$$K = U_K \cdot S_K, \quad V = U_V \cdot S_V$$

其中, U_K, U_V 是低秩矩阵, S_K, S_V 是压缩矩阵。

• 联合压缩:

- 对压缩后的键和值矩阵进行联合压缩，进一步减少存储需求。

• 潜在注意力计算:

- 在压缩后的表示上进行注意力计算。

• 公式表示:

$$\text{MLA}(Q, K, V) = \text{softmax} \left(\frac{Q(U_K S_K)^T}{\sqrt{d_k}} \right) (U_V S_V)$$

• 优势:

- 显著降低推理过程中的内存占用。
- 保持模型性能的同时，提高推理效率。

二、更大规模的参数

2.1 参数规模

DeepSeek 的 LLM 拥有数百亿到数千亿参数，例如：

- DeepSeek-V3: 拥有 6710 亿参数。

- **DeepSeek-LLM**: 在 2 万亿 token 的数据集上进行训练。
- **DeepSeek-V2**: 在 8.1 万亿 token 的多样化语料库上预训练。

2.2 关键技术

(1) 分布式训练

- **数据并行**:
 - 将训练数据分配到多个计算节点上，每个节点独立计算梯度，最后进行梯度聚合和参数更新。
 - **优点**: 提高数据吞吐量，加快训练速度。
- **模型并行**:
 - 将模型参数分配到多个计算节点上，每个节点负责计算模型的一部分参数。
 - **优点**: 解决单个节点的显存限制问题，支持训练更大规模的模型。
- **流水线并行**:
 - 将模型的不同层分配到不同的计算节点上，实现流水线式的并行计算。
 - **优点**: 提高计算资源利用率，减少计算等待时间。

(2) 混合精度训练

- **半精度浮点数**:
 - 使用半精度（FP16）浮点数表示模型参数和梯度。
 - **优点**: 减少显存占用和计算量。
- **损失缩放**:
 - 对损失值进行缩放，以避免梯度消失。
 - **原理**: 半精度浮点数表示范围有限，损失缩放可以防止梯度值过小，导致梯度消失。

(3) 混合专家模型 (MoE)

- **作用:** 通过 MoE 架构, DeepSeek 可以有效管理大规模参数, 避免计算资源浪费。
- **具体实现:**
 - 每个 token 仅激活部分专家网络, 而不是所有专家网络。
 - 例如, DeepSeek-V3 的每个 token 仅激活 370 亿参数, 而不是全部 6710 亿参数。

(4) 模型压缩与剪枝

- **权重共享 (Weight Sharing) :**
 - 共享模型参数, 例如在 Transformer 中共享参数矩阵。
 - **优点:** 减少参数数量, 降低存储和计算需求。
- **剪枝 (Pruning) :**
 - 删除模型中不重要的参数或神经元, 例如:
 - **权重剪枝:** 删除权重值较小的参数。
 - **神经元剪枝:** 删除激活值较低的神经元。
 - **优点:** 进一步减少参数数量, 提高模型效率。

三、过程模型公式

3.1 稀疏注意力机制

- **局部注意力:**

LocalAttention(Q, K, V) = softmax $\left(\frac{QK_{local}^T}{\sqrt{d_k}} \right) V_{local}$

CSDN @爱研究的小牛

- **块状注意力:**

$$\text{BlockwiseAttention}(Q, K, V) = \text{softmax} \left(\frac{QK_{block}^T}{\sqrt{d_k}} \right) V_{block}$$

- 随机注意力:

$$\text{RandomAttention}(Q, K, V) = \text{softmax} \left(\frac{QK_{random}^T}{\sqrt{d_k}} \right) V_{random}$$

- 组合策略:

$$\text{SparseAttention}(Q, K, V) = \alpha \cdot \text{LocalAttention} + \beta \cdot \text{BlockwiseAttention} + \gamma \cdot \text{RandomAttention}$$

其中, α, β, γ 是权重系数。

3.2 混合专家模型 (MoE)

- 门控网络:

$$\text{Gate}(x) = \text{softmax}(xW_g + b_g)$$

- 专家网络输出:

$$\text{Expert}_i(x) = \text{ExpertNetwork}_i(x)$$

- 最终输出:

$$y = \sum_{i=1}^N \text{Gate}_i(x) \cdot \text{Expert}_i(x)$$

3.3 多头潜在注意力 (MLA)

- 低秩分解:

$$K = U_K \cdot S_K, \quad V = U_V \cdot S_V$$

- 潜在注意力计算:

$$\text{MLA}(Q, K, V) = \text{softmax} \left(\frac{Q(U_K S_K)^T}{\sqrt{d_k}} \right) (U_V S_V)$$

四、训练方法

4.1 预训练

- **目标:** 让模型学习语言的通用表示和知识。
- **方法:** 使用海量的文本数据进行训练，模型通过预测文本序列中的下一个词或字符来学习语言模式。
- **数据:** 包含互联网上的各种文本数据，如书籍、文章、网页等。
- **训练目标:** 例如，**掩码语言模型（Masked Language Model, MLM）**任务，即随机遮蔽文本中的部分词，让模型预测被遮蔽的词。

4.2 监督微调

- **目标:** 针对特定任务或应用场景，对预训练模型进行微调。
- **方法:** 使用特定任务的数据集对模型进行训练，例如：
 - **文本分类:** 对文本进行分类，例如情感分析、主题分类等。
 - **问答系统:** 根据用户的问题生成相应的答案。
 - **机器翻译:** 将一种语言翻译成另一种语言。
- **训练目标:** 根据具体任务设计相应的损失函数，例如交叉熵损失（Cross-Entropy Loss）。

4.3 人类反馈的强化学习

- **目标:** 利用人类反馈信息，进一步优化模型输出。

- **方法:** 将人类对模型输出的反馈作为奖励信号，使用强化学习算法调整模型参数。
- **优势:** 使模型生成的结果更符合人类的期望和需求，提高模型的质量和实用性。